```
  class Product < ApplicationRecord
»   has_many :line_items

»   before_destroy :ensure_not_referenced_by_any_line_item

    #...


»   private

»     # ensure that there are no line items referencing this product
»     def ensure_not_referenced_by_any_line_item
»       unless line_items.empty?
»         errors.add(:base, 'Line Items present')
»         throw :abort
»       end
»     end
  end
```

Here we declare that a product has many line items and define a *hook* method named `ensure_not_referenced_by_any_line_item`. A hook method is a method that Rails calls automatically at a given point in an object's life. In this case, the method will be called before Rails attempts to destroy a row in the database. If the hook method throws `:abort`, the row isn't destroyed.

Note that we have direct access to the `errors` object. This is the same place that the `validates` method stores error messages. Errors can be associated with individual attributes, but in this case we associate the error with the base object.

Before moving on, add a test to ensure that a product in a cart can't be deleted:

rails7/depot_f/test/controllers/products_controller_test.rb

```
» test "can't delete product in cart" do
»   assert_difference("Product.count", 0) do
»     delete product_url(products(:two))
»   end
»
»   assert_redirected_to products_url
» end

  test "should destroy product" do
```